Transformation and Optimization in Deep Learning for Trajectory Analysis

Alphy Devachen and Vinayak Devesan

Sixth Semester IMSC Project

Under the Guidance
Dr. Linu Pinto
Assistant Professor
DEPARTMENT OF MATHEMATICS
Cochin University of Science and Technology

August 14, 2024

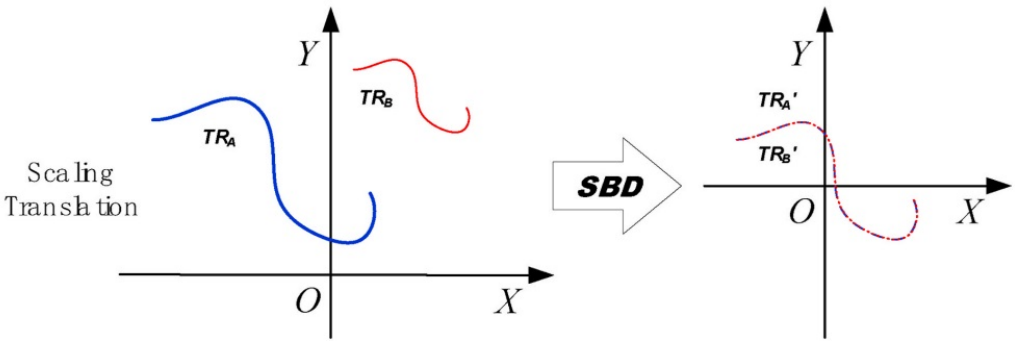
Project Description

- The Project focus on the analysis of Tragectories ,which is adapted from the paper "An algorithm for extracting similar segments of moving target trajectories based on shape matching "
- Mathematical Concepts like distance metrics, geometric transformations and Gradients are applied to classify the images based on the tragectory analysis.
- With advancements in technology like satellite positioning, drones, road cameras, and various tracking systems, location data on moving targets such as people, vehicles, ships, etc can be extensively collected.
- This data, collected over time with timestamps, forms a trajectory.

Objectives

Key Objectives:

- **Distance Metrics:** Utilize various distance metrics to quantify and compare trajectories, identify similarity patterns, and detect anomalies.
- **Geometric Transformations:** Apply transformations such as translation and scaling to normalize and standardize trajectories. This ensures that trajectories from different sources or times are comparable.
- **Gradient:** Gradient approaches enables the classification of similar trajectories in a group.



Applications of Trajectory Analysis

Applications:

- **Urban Traffic Management:** Identify traffic congestion points and common vehicle routes for improved road condition forecasting and urban planning.
- **Wildlife Tracking:** Analyze animal migration patterns to understand behavior and environmental impacts.
- **Maritime Navigation:** Enhance maritime safety and route planning by analyzing vessel trajectories. By analyzing vessel trajectories, maritime navigation systems can enhance safety by predicting collisions and identifying hazards, and optimize routes for efficiency by using historical and real-time data. This improves traffic management, port operations, and overall maritime safety and efficiency.
- **In-Store Movement Tracking**: Analyze customer movement patterns within retail stores to optimize store layouts, improve product placement, and enhance shopping experiences.

Trajectory

Trajectory with n Points:
A trajectory, denoted as TR, is a sequence of n points that represent the path of a moving object. Each point in the trajectory is identified as p_j, where j ranges from 1 to n. This can be mathematically represented as:

$$\mathrm{TR} = (p_1, p_2, \ldots, p_n)$$

Representation of Trajectory in XY Coordinates:
Each point p_j in the trajectory can be expressed in terms of its XY coordinates. Specifically, point p_j is represented as:

$$p_j = (x_{p_j}, y_{p_j}) \in \mathbb{R}^2$$

where x_{p_j} and y_{p_j} are the X and Y coordinates of point p_j, respectively.

Resampling Trajectory

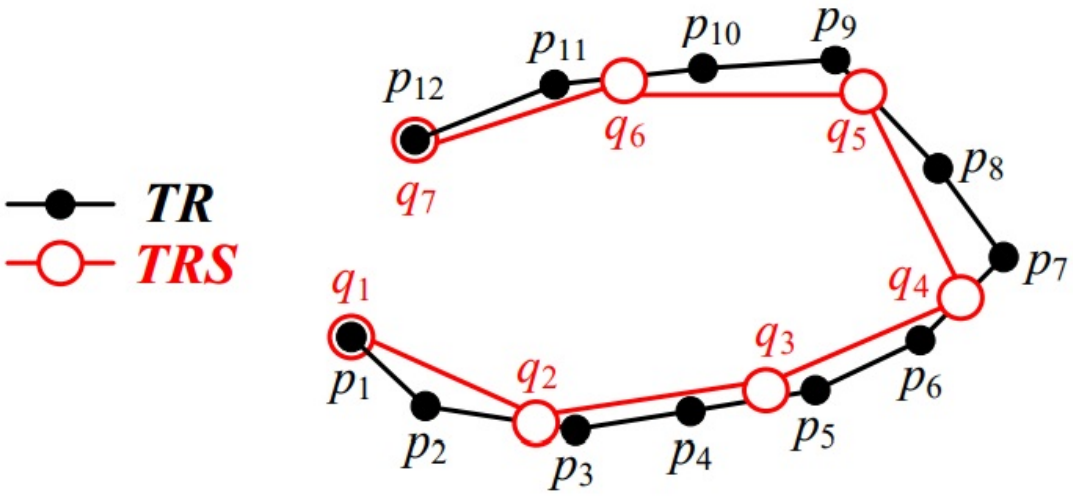
Resampling Points:
The process of resampling involves adjusting the original trajectory, which consists of n points, to a new trajectory with m points. This transformation aims to create a new trajectory, denoted as TRS, where each new point is evenly spaced or adjusted according to a specific criterion. The new trajectory can be expressed as:

$$\text{TRS} = (q_1, q_2, \ldots, q_m)$$

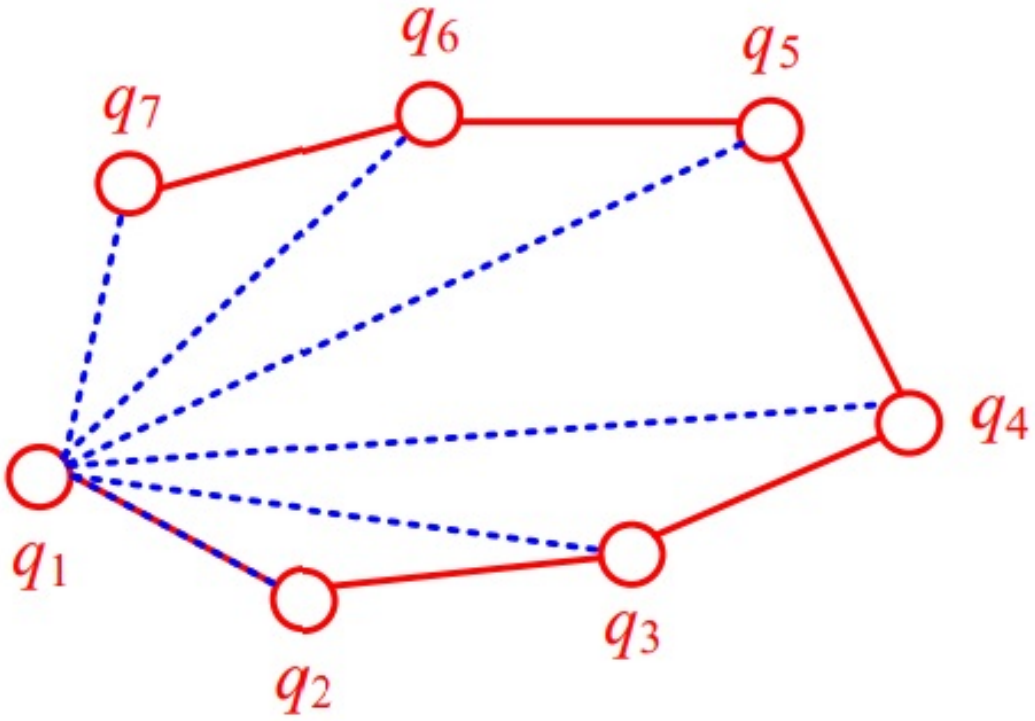
Representation in XY Coordinates:
Each new point q_j in the resampled trajectory is also represented in XY coordinates. Specifically, point q_j is represented as:

$$q_j = (x_{q_j}, y_{q_j})$$

where x_{q_j} and y_{q_j} are the X and Y coordinates of the new point q_j, respectively.


TR
TRS
p_{12}
p_{11}
p_{10}
p_9
q_6
q_5
p_8
q_7
p_7
q_1
q_4
q_3
p_1
q_2
p_6
p_2
p_3
p_4
p_5

(a)

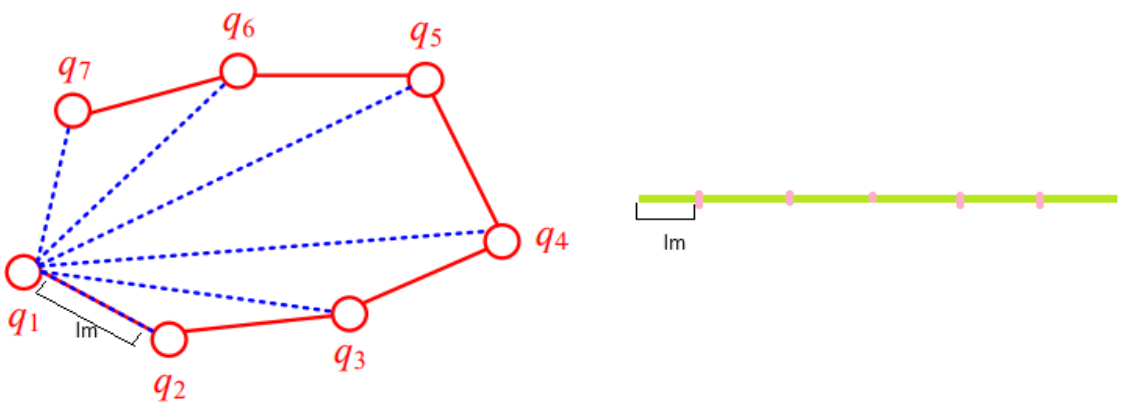

q_6
q_5
q_7
q_4
q_1
q_3
q_2

Length of the trajectory and Equidistant Sampling Length

- The total length of the trajectory L is the sum of the Euclidean distances between consecutive points in the trajectory.

$$L = \sum_{j=1}^{n-1} \|p_{j+1} - p_j\|$$

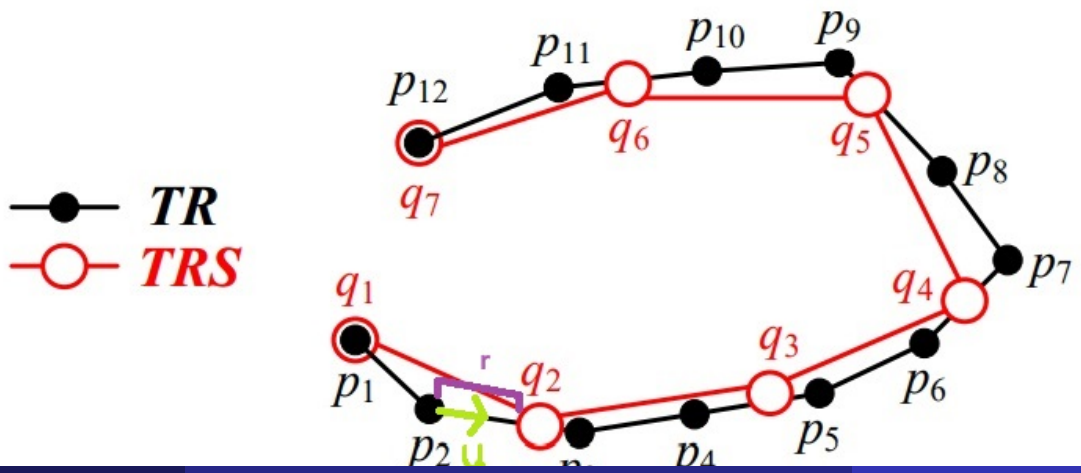
- The trajectory length L is divided into $m-1$ segments for resampling.
- The length of each segment in the resampled trajectory is $l_m = \frac{L}{m-1}$, ensuring equidistant points in the new trajectory.



Trajectory Deformation

Assume a trajectory with points $p_1, p_2, p_3, \ldots, p_{12}$:

1. Start from an initial point $q_1 = p_1$.
2. Start with $k = 2$: Calculate the cumulative distance from p_1 to p_2.
3. If the distance is less than l_m, increment k and repeat.
4. Once the cumulative distance exceeds l_m, identify the segment between p_k and p_{k+1}.
5. Place q_2: Use the remaining distance $r = l_m - cd(p_{k-1}, p_k)$ to place q_2 between p_k and p_{k+1} by pushing r distance from p_k in the direction u.
6. Same procedure is applied till q_7



- **Conditions for Equidistant vectors can be Generalized as follows:**

$$\sum_{j=1}^{k} \|p_{j+1} - p_j\| \leq l_m < \sum_{j=1}^{k+1} \|p_{j+1} - p_j\| \quad \text{where } 2 \leq k \leq n-1$$

- The summation $\sum_{j=1}^{k+1} \|p_{j+1} - p_j\|$ represents the cumulative distance from the initial point p_1 to the point p_{k+1}.
- The condition $\sum_{j=1}^{k+1} \|p_{j+1} - p_j\| > l_m$ ensures that the total distance exceeds the desired equidistant interval l_m.
- The point q_2 is placed between p_k and p_{k+1} based on the remaining distance $l_m - \sum_{j=1}^{k-1} \|p_{j+1} - p_j\|$.

Normalization and Scaling Transformations for Resampling

- Let $p_k = (x_{p_k}, y_{p_k})$ and $p_{k+1} = (x_{p_{k+1}}, y_{p_{k+1}})$ be the original points in the trajectory.
- We want to place the new point $q_2 = (x_{q_2}, y_{q_2})$ such that it maintains the resampled distance l_m.
- The remaining distance to be covered along the segment from p_k to p_{k+1} is given by:

$$lm - \sum_{j=1}^{k-1} \|p_{j+1} - p_j\|$$

- This sum represents the cumulative distance from the start to p_k, subtracted from lm, the desired equidistant interval.

- Let $\mathbf{v}$ denotes the vector that originates at point p_k and points towards p_{k+1}. It describes the direction and magnitude of the straight line connecting these two points.
- The length of the vector $\mathbf{v}$, is the Euclidean distance between the points p_k and p_{k+1}. It is calculated as:

$$\|\mathbf{v}\| = \|p_{k+1} - p_k\| = \sqrt{(x_{p_{k+1}} - x_{p_k})^2 + (y_{p_{k+1}} - y_{p_k})^2}$$

- The direction of $\mathbf{v}$ can be described by a unit vector $\hat{u}$, which is $\mathbf{v}$ normalized to have a length of 1.

$$\hat{u} = \frac{\mathbf{v}}{\|\mathbf{v}\|} = \left(\frac{x_{p_{k+1}} - x_{p_k}}{\|p_{k+1} - p_k\|}, \frac{y_{p_{k+1}} - y_{p_k}}{\|p_{k+1} - p_k\|} \right)$$

Transformation of vectors in Resampling along the direction of $\hat{u}$

- Now, the coordinates of q_2 can be found by moving from p_k by the remaining distance(r) along the direction of $\hat{u}$:

$$(x_{q_2}, y_{q_2}) = (x_{p_k}, y_{p_k}) + r.\hat{u}$$

- Transformed vector,

$$(x_{q_2}, y_{q_2}) = (x_{p_k}, y_{p_k}) + \left(lm - \sum_{j=1}^{k-1} \|p_{j+1} - p_j\| \right) \cdot \frac{(x_{p_{k+1}} - x_{p_k}, y_{p_{k+1}} - y_{p_k})}{\|p_{k+1} - p_k\|}$$

Trajectory Analysis based on Translation and Rotation invariance property

Translation in $\mathbb{R}^2$ is a geometric transformation that moves every vector in $\mathbb{R}^2$ to a constant distance in a specified direction. The translation of a point (x, y) by a vector (a, b) results in a new point (x', y'), which can be defined mathematically as:

$$(x', y') = (x + a, y + b)$$

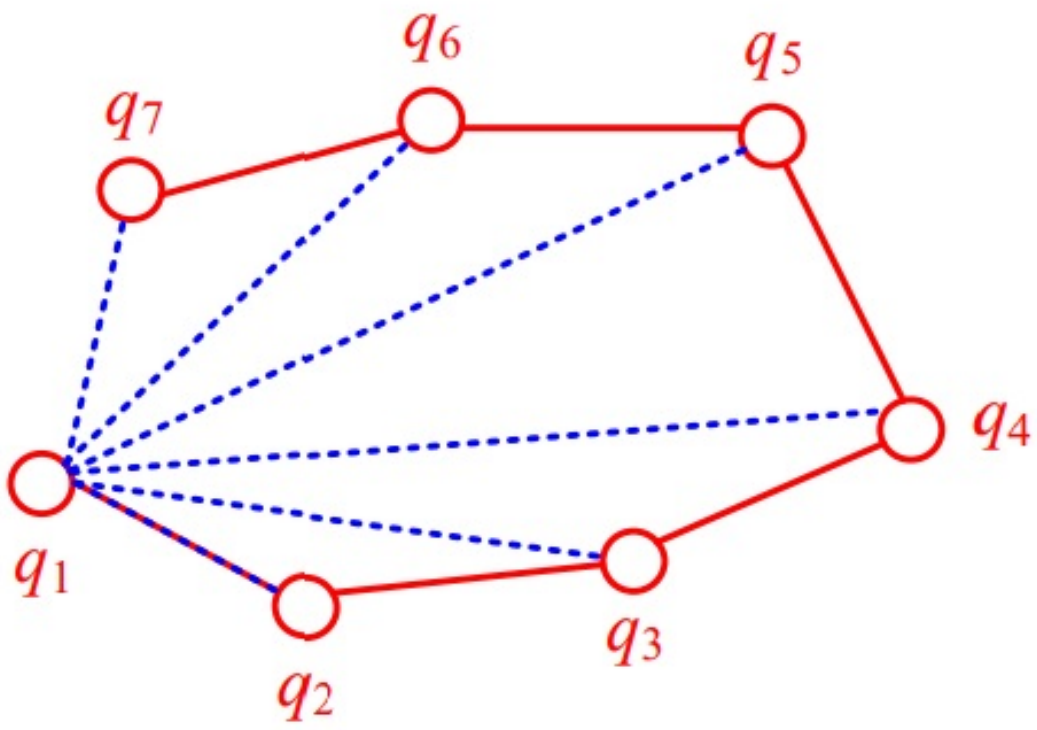
A **scaling** transformation in $\mathbb{R}^2$ is defined by scaling factors a and b applied to the x- and y-coordinates, respectively. The scaling transformation of a point (x, y) results in a new point (x', y'), which can be mathematically defined as:

$$(x', y') = (a \cdot x, b \cdot y)$$

where:

- a is the scaling factor for the x-coordinate.
- b is the scaling factor for the y-coordinate.

- Resampled trajectory is a polygon with m vertices $q_1, q_2, \ldots, q_m$.
- The polygon can be divided into $m-2$ triangles.
- For each triangle, one vertex is fixed (e.g., q_1), and the other two vertices vary sequentially. The k-th triangle can be denoted as $\Delta q_1 q_{k+1} q_{k+2}$, where k varies from 1 to $m-2$.



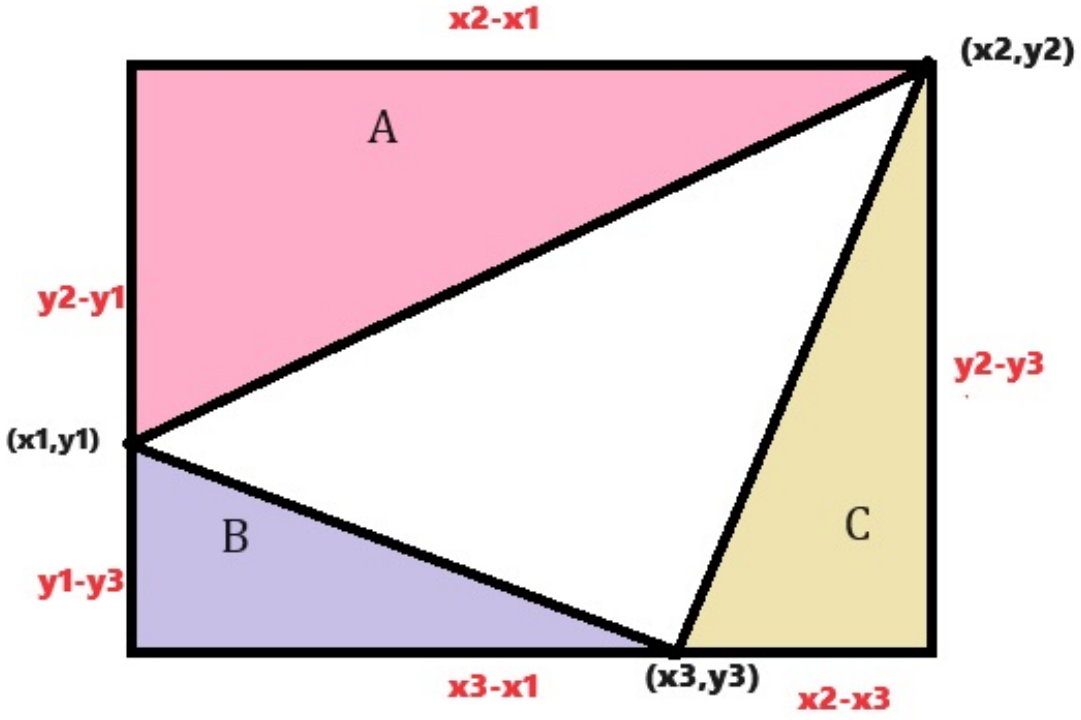
- Mathematically, for a triangle with vertices (x_{q_1}, y_{q_1}), (x_{q_2}, y_{q_2}), and (x_{q_3}, y_{q_3}), the centroid $C_1 = (x_{c_1}, y_{c_1})$ is given by the average of the vertices' coordinates:

$$(x_{c_1}, y_{c_1}) = \left(\frac{x_{q_1} + x_{q_2} + x_{q_3}}{3}, \frac{y_{q_1} + y_{q_2} + y_{q_3}}{3} \right)$$

- Let C_k denote the centroid of the k-th triangle.
- The area of the k-th triangle is denoted by S_k.
- When computing the centroid of a polygon, each triangle's contribution to the overall centroid should be proportional to its area. Larger triangles should have a greater influence on the centroid than smaller triangles. This is achieved by taking a weighted average of the centroids of the individual triangles, with the weights being the areas of those triangles.
- Hence the weighted centroid of the polygon can be found by

$$C = (x_c, y_c) = \frac{\sum_{k=1}^{m-2} S_k C_k}{\sum_{k=1}^{m-2} S_k}$$

Evaluating the area of each triangle S_k



- Area (A)$=\frac{1}{2}(x_2 - x_1)(y_2 - y_1)$
- Area (B)$=\frac{1}{2}(x_2 - x_3)(y_2 - y_3)$
- Area (C)$=\frac{1}{2}(x_3 - x_1)(y_1 - y_3)$
- Area (rectangle)$=(x_2 - x_1)(y_2 - y_3)$

- Area of triangle=Area(rectangle)-[Area (A)+Area (B)+Area (C)]

$=(x_2 - x_1)(y_2 - y_3) - [\frac{1}{2}(x_2 - x_1)(y_2 - y_1) + \frac{1}{2}(x_2 - x_3)(y_2 - y_3) + \frac{1}{2}(x_3 - x_1)(y_1 - y_3)]$

$=x_2y_2 - x_2y_3 - x_1y_2 + x_1y_3 - \frac{1}{2}[x_2y_2 - x_2y_1 - x_1y_2 + x_1y_1 + x_2y_2 - x_2y_3 - x_3y_2 + x_3y_3 + x_3y_1 - y_3x_3 - x_1y_1 + x_1y_3]$

$$= -\frac{1}{2}[x_1(y_2 - y_3) - x_2(y_1 - y_3) + x_3(y_1 - y_2)] = -\frac{1}{2}\begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$$

- Hence area of each triangle is

$$S_k = \frac{1}{2} \mid x_{q_1}(y_{q_k} - y_{q_{k+1}}) + x_{q_k}(y_{q_{k+1}} - y_1) + x_{q_{k+1}}(y_1 - y_{q_k}) \mid$$

- Total area of polygon is: $S = \sum_{k=1}^{m-2} S_k$

Translation and Scaling

- Translation of origin to the centroid is achieved by subtracting the centroid coordinates from each point:

$$\boxed{(x'_{q_i}, y'_{q_i}) = (x_{q_i} - x_c, y_{q_i} - y_c)}$$

- If features have drastically different scales, the distance calculation can be heavily skewed by the features with larger ranges. These new translated points above are scaled and normalized in such a way that the influence of the area of the polygon is minimized:

$$(x''_{q_i}, y''_{q_i}) = \left(\frac{x'_{q_i}}{S}, \frac{y'_{q_i}}{S}\right)$$

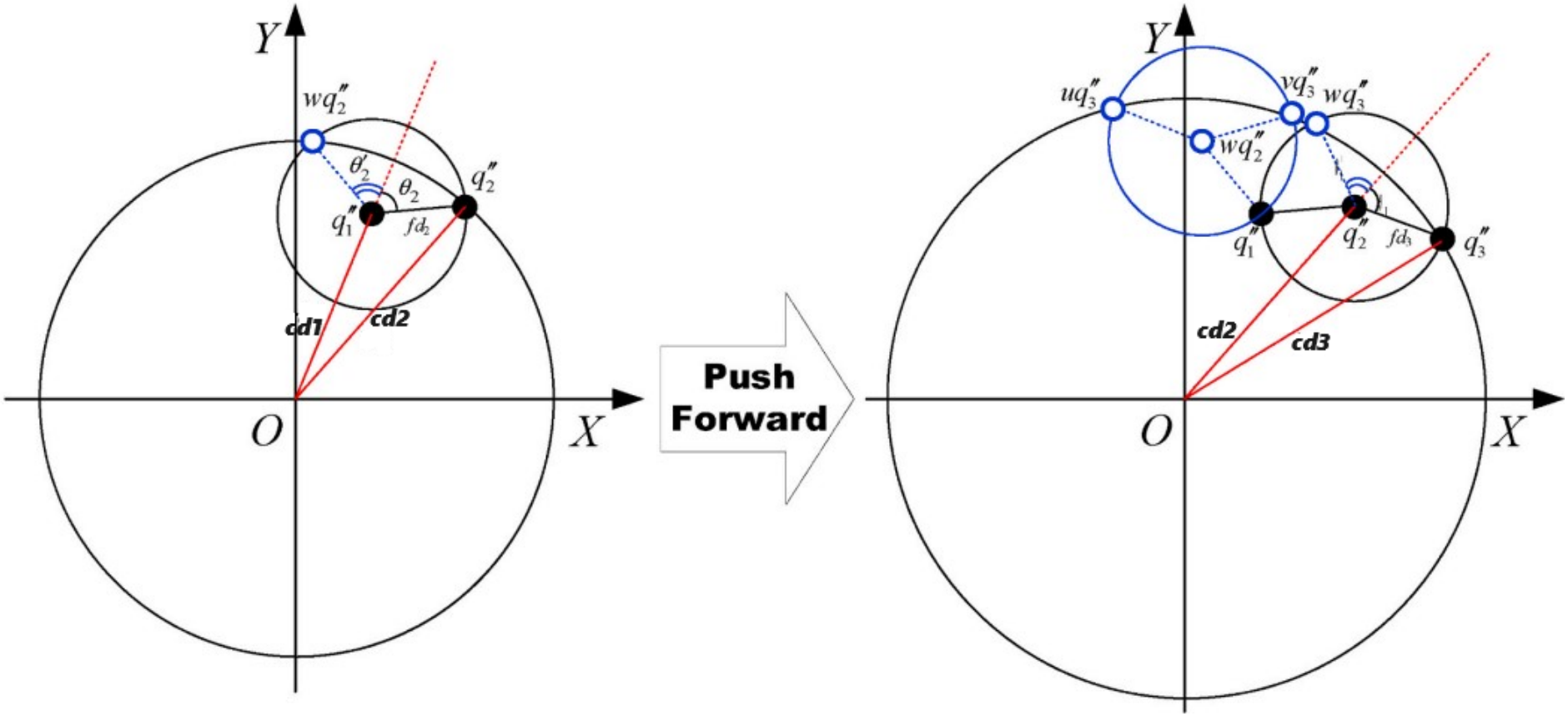
Describing Trajectory Shape

- From the coordinates $q_i'' = (x_{q_i}'', y_{q_i}'')$, which represent each point of the resampled trajectory, we can describe the shape of the trajectory using two features: the cd_i and the fd_i.
- cd_i is the Euclidean distance between each q_i and the centroid of the polygon. Since the centroid of the polygon is at the origin $(0, 0)$ after our translation, the cd_i is given by the formula:

$$cd_i = \sqrt{(x_{q_i}'' - 0)^2 + (y_{q_i}'' - 0)^2}$$

- fd_i is the forward distance:
  - For $i = 1$, $fd_i = 0$ (since only a single point is available there, the distance is zero)
  - For $i \geq 2$, $fd_i = \left\| q_i'' - q_{i-1}'' \right\|$
- Now the new trajectory can be represented using the set, $Z = \{(cd_1, fd_1), (cd_2, fd_2), \ldots, (cd_m, fd_m)\}$
- Trajectory is described in xy coordinate system in terms of distance only, but no direction has been assigned. Hence the representation of this trajectory is not unique.

Non-Uniqueness in Trajectory



Non-Uniqueness of the Trajectory

- We start by fixing the centroid of the polygon at the origin $(0, 0)$.
- The first point q_1 is defined by its distance cd_1 from the centroid. Since q_1 is the first point, we draw cd_1 in any direction from the origin. There is no restriction on the direction, so we can choose any point on a circle of radius cd_1 centered at the origin.
- The second point q_2 is defined by its distance cd_2 from the centroid and the distance fd_2 from q_1. To find q_2:
  - Draw a circle centered at the origin with radius cd_2.
  - From q_1, measure a distance fd_2. This creates two possible points where q_2 can be, as the line segment of length fd_2 intersects the circle at two points.

Non Uniqueness

- The third point q_3 is defined by its distance cd_3 from the centroid and the distance fd_3 from q_2. To find q_3:
  - Draw a circle centered at the origin with radius cd_3.
  - From each possible position of q_2, measure a distance fd_3. Each q_2 has two possible points for q_3, resulting in four possible positions for q_3.
- This process continues for each subsequent point q_i:
  - Draw a circle centered at the origin with radius cd_i.
  - From each possible position of q_{i-1}, measure a distance fd_i. This doubles the number of possible positions for each new point.
- By the time we reach the m-th point q_m, there are 2^{m-1} possible positions for q_m.
- **Conclusion:** Because each point can have multiple possible positions based on the given distances, there are many possible ways to construct the trajectory. This leads to non-uniqueness, meaning the same shape can correspond to multiple different trajectories.

Unique Trajectory Representation

- To make the trajectory unique, the method involves using a deflection angle, denoted as θ_i.
- θ_i is the angle measured clockwise from the direction $q''_{i-1}q''_i$ (the line connecting two successive points on the trajectory) to the line Oq''_{i-1} (a reference direction from a point O to the previous point q''_{i-1}).
- The angle θ_i is constrained within the range $[0, 2\pi)$, meaning it can take any value from 0 to just below 2π radians.
- When the deflection angle θ_i is between 0 and π (i.e., the first half of the circle), the component scd_i is assigned a negative sign, making it $-cd_i$.
- $\theta_i \in [\pi, 2\pi)$: When the deflection angle θ_i is between π and 2π (i.e., the second half of the circle), scd_i retains its positive sign.
- The signed component scd_i, after applying the above sign convention, is denoted as scd_i. This signing process ensures that each trajectory segment has a consistent and unique representation.

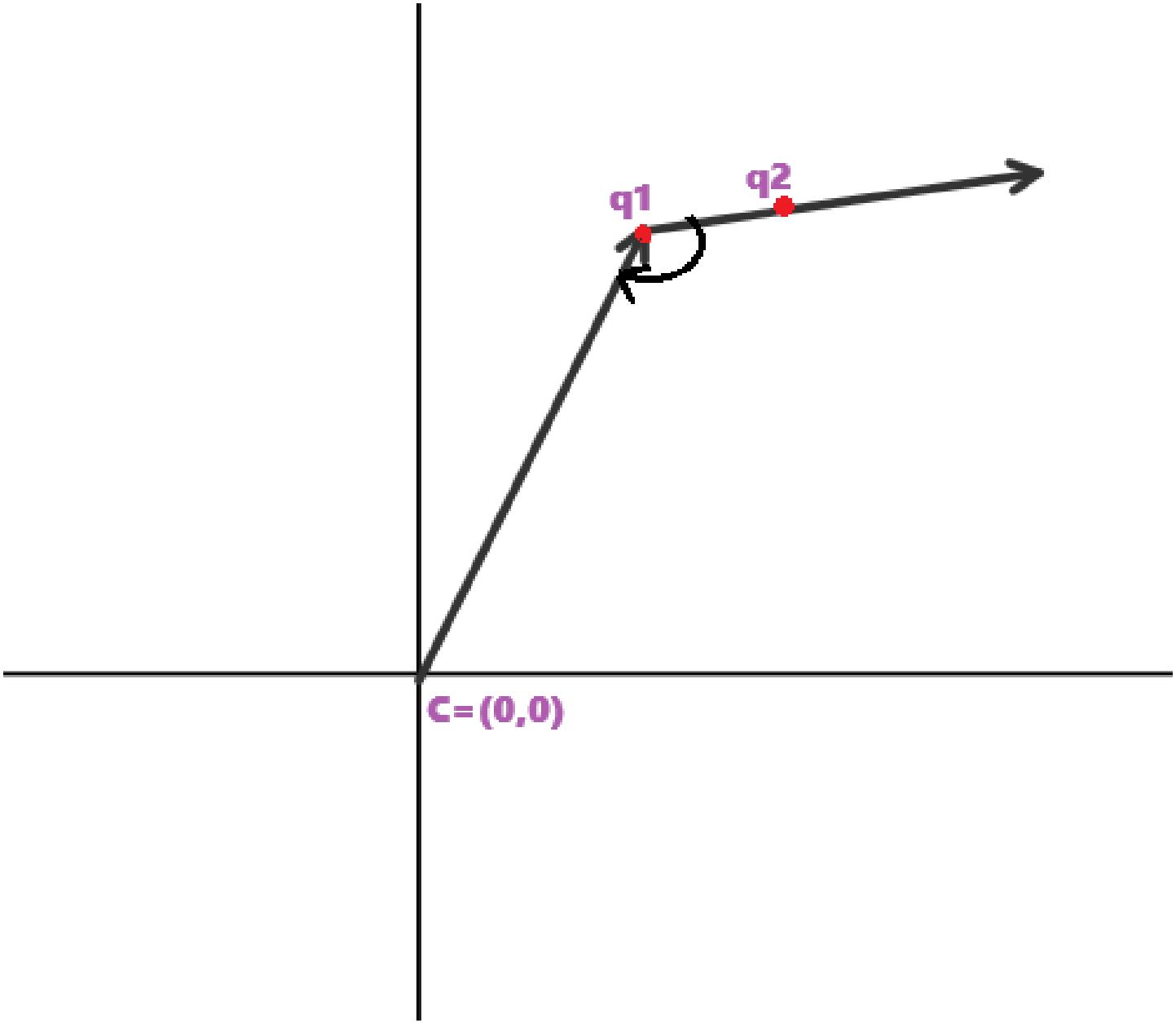
- It is assumed that the signs of sd_1 and sd_2 (the signed components corresponding to the first two points) are the same. This assumption might be to maintain consistency and avoid ambiguity in the trajectory's initial direction.
- The trajectory is uniquely defined by a set C, which consists of a sequence of ordered pairs. Specifically,

$$C = \{(scd_1, fd_1), (scd_2, fd_2), \ldots, (scd_m, fd_m)\}$$

  This can also be represented by the sequence

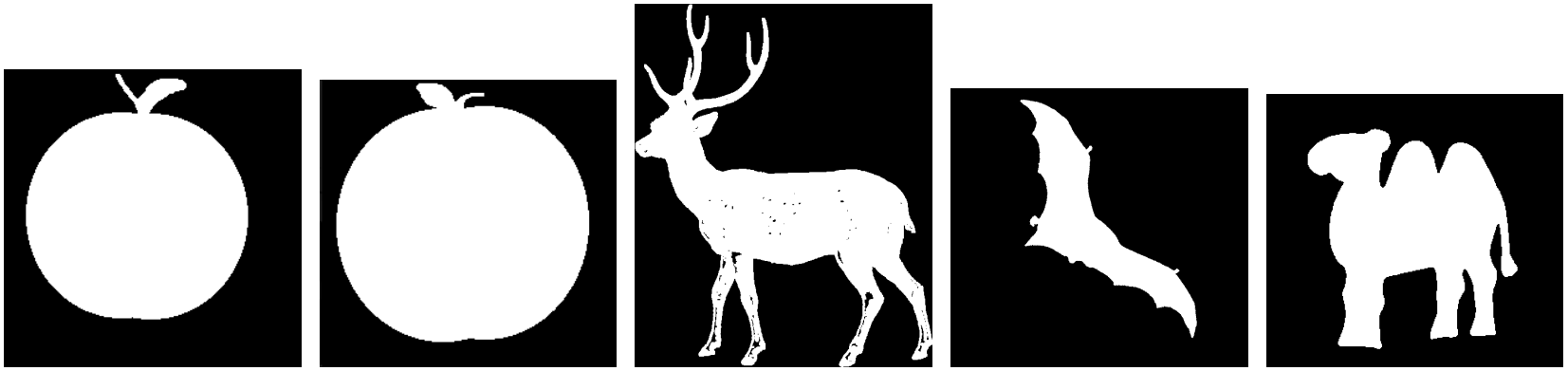
$$(SF_1, SF_2, \ldots, SF_m)$$

  where SF_i denotes the ordered pair (scd_i, fd_i) at index i.
- Each SF_i represents the pair (scd_i, fd_i), where scd_i is the signed component and fd_i is the other component describing q_i''.
- The index i runs from 1 to m, representing different points on the trajectory.
- This set C uniquely identifies the trajectory based on the described angle and sign conventions.


q1
q2
C=(0,0)

Construction of Trajectory from the dataset

- We are using MPEG-7 data set.This data set consists of black and white images, boundary of each image can be viewed as a trajectory.
- Here each Trajectory T_i is a set of 100 points, $T_i = \{q''_1, q''_2, \cdots, q''_n\}_i$ where $n = 100$ and $q''_j = (x''_{q_j}, y''_{q_j}) \in \mathbb{R}^2$.
- Hence to perform the optimization, each trajectory T_i corresponding to an image i in the dataset can be viewed as a vector v_i of dimension $2n$ (In this case we have dimension as 200).



- Using the trajectory construction as described above, a set of vectors, each with dimension $m = 200$, is considered as input data.
- Hence, mathematically we can represent the input data as:

$$\mathbf{X} = v_i = (v_{i1}, v_{i2}, \ldots, v_{im}), v_{ij} = \begin{cases} x''_{q_j}, & \text{if } k = 2j - 1 \\ y''_{q_j}, & \text{if } k = 2j \end{cases}, v_{ij} \in \mathbb{R}, 1 \leq j \leq 100\}$$

- Consider another set $\mathbf{Y}$, called the target data set, defined mathematically as the set consisting of elements from the standard basis of dimension $r = 70$.
- Labels of the images are converted to some element in the standard basis of dimension $r = 70$. For example:
  - Apple $\rightarrow [1, 0, 0, 0, \ldots]_{1\times 70}$
  - Bat $\rightarrow [0, 1, 0, 0, \ldots]_{1\times 70}$

Mathematical Model for Optimization

- The objective is to construct a continuously differentiable function $f : \mathbf{X} \to \mathbf{Y}$ such that $f(v_i) = y_i, \ \forall i = 1, 2, ...N$. Here $v_i \in \mathbb{R}^{200}$, $y_i \in \mathbb{R}^{70}$ and each $y_i = e_j \in \mathbf{Y}$ for some j. In an application perspective we say the function f is a mathematical model that maps each elements in the set $\mathbf{X}$ to the corresponding targets in the set $\mathbf{Y}$.
- Here the proposed Mathematical Model for the optimization is a non linear function $f(v_p, \vec{w})$
- The function $f(v_p, \vec{w})$ is the final output of the neural network for p^{th} input vector $v_p \in \mathbb{R}^{200}$ and is given by $f(v_p, \vec{w}) = [a_{1p}, a_{2p}, \cdots a_{n_2 p}]$, where each element

$$a_{kp} = \frac{e^{\sum_{i=1}^{n_1} w_{ki}^{(2)} \tanh\left(\sum_{j=1}^{n_0} w_{ij}^{(1)} x_{jp}\right)}}{\sum_{l=1}^{n_2} e^{\sum_{i=1}^{n_1} w_{li}^{(2)} \tanh\left(\sum_{j=1}^{n_0} w_{ij}^{(1)} x_{jp}\right)}}$$

- Here $n_0 = 200, n_1 = 128, n_2 = 70$

Objective Function to minimize for Optimization

- Define the objective function as the square of the L_2 norm of $\mathbf{Y} - f(v_p, \vec{w})$:
$$L(\vec{w}) = \|\mathbf{Y} - f(v_p, \vec{w})\|_2^2$$
- Hence the goal is to find the optimal function $f(v_p, \vec{w})$ such that the objective function $L(\vec{w})$ is minimized.
- The optimal weight vector w^* that minimizes the function $L(\vec{w})$ is given by:
$$\vec{w}^* = \arg\min_{\vec{w}} \|\mathbf{Y} - f(v_p, \vec{w})\|_2^2$$

Taylor series and Optimization

- The first-order Taylor series expansion of $L(\vec{w})$ around $\vec{w}_0$ is given by:

$$L(\vec{w}_0 + \Delta\vec{w}) \approx L(\vec{w}_0) + \nabla L(\vec{w}_0)^T \Delta\vec{w}$$
$$L(\vec{w}_0 + \Delta\vec{w}) - L(\vec{w}_0) \approx \nabla L(\vec{w}_0)^T \Delta\vec{w}$$
$$\Delta L \approx \nabla L(\vec{w}_0)^T \Delta\vec{w}_0$$

- we need to move in the direction of the fastest decrease of the function $L(\vec{w})$.
- The direction in which the change in weights $\Delta\vec{w}$ should occur is where the function decreases at the maximum rate.
- The change in the function value ΔL is proportional to the dot product of the gradient ∇L and the change in weights $\Delta\vec{w}$, scaled by the cosine of the angle θ between them:

$$\Delta L = \|\nabla L\| \cdot \|\Delta\vec{w}\| \cdot \cos\theta$$

- ΔL decreases the most when the angle θ is π, meaning the direction of $\Delta\vec{w}$ is directly opposite to the gradient vector ∇L.
- Therefore, the change in weights $\Delta\vec{w}$ should be a negative scalar multiple of the gradient ∇L:

$$\Delta\vec{w} = -h\nabla L$$

  where h is a positive scalar (Technically called as the learning rate).
- $\nabla L(\vec{w}_0)$ is the gradient of $L(\vec{w})$ at $\vec{w}_0$, and $\Delta\vec{w}$ represents a small change in the weights.
- The term $\nabla L(\vec{w}_0)^T \Delta\vec{w}_0$ represents the linear approximation of how $L(\vec{w})$ changes with respect to $\vec{w}$.
- To minimize $L(\vec{w})$,we want this term to be as negative as possible, meaning you want $\Delta\vec{w}$ to be in the direction that makes $\nabla L(\vec{w}_0)^T \Delta\vec{w}$ most negative.
- This occurs when $\Delta\vec{w}$ is in the opposite direction to $\nabla L(\vec{w}_0)$.

- Mathematically:

$$\Delta \vec{w} = -h \nabla L(\vec{w}_0)$$

This corresponds to moving in the direction of steepest descent, which is the direction where the function decreases at the fastest rate.

- Since $\vec{w}$ should be changed in the direction of steepest descent, the weight update formula for performing iteration steps, such that after each iteration the weights move towards the local minimum, is given by:

$$\vec{w}_{\text{new}} = \vec{w}_{\text{old}} - h \nabla L$$

- $\vec{w}_{\text{old}}$ represents the current weights, $\vec{w}_{\text{new}}$ represents the updated weights after the iteration, h is a positive scalar known as the learning rate, and ∇L is the gradient of the loss function L with respect to the weights $\vec{w}$.

- The formula ensures that the weights are adjusted in the direction opposite to the gradient, which corresponds to the direction of maximum decrease of the loss function.
- By iteratively applying this update rule, the weights move closer to the local minimum of the loss function L.
- The entire iterative procedure is being performed in the neural network structure.
- The performance in the optimization are evaluated using a concept called **Accuracy**.

Measure to evaluate Optimization of $L(\vec{w})$

- The output of the neural network before applying softmax is a vector of z_{kp} values for each class k (where $k = 1, 2, \ldots, n_2$, and n_2 is the number of classes, which is 70 in this case). The softmax function converts these values into probabilities by:

$$\hat{y}_{kp} = \frac{e^{z_{kp}}}{\sum_{l=1}^{n_2} e^{z_{lp}}}$$

  where $\hat{y}_{kp}$ is the predicted probability for class k given input v_p.
- The predicted class $\hat{y}_p$ for input v_p is the class with the highest probability:

$$\hat{y}_p = \arg\max_k \hat{y}_{kp}$$

- The true target output y_p is often represented as a one-hot encoded vector. For example, if there are 70 classes and the correct class for the p-th input is class j, then:

$$y_p = [0, 0, \ldots, 1, \ldots, 0] \quad \text{(with 1 at the } j\text{-th position)}$$

  This means $y_p = e_j$ for some j, where e_j is the standard basis vector with 1 in the j-th position and 0 elsewhere.
- The prediction is correct if the index of the highest probability in $\hat{y}_p$ matches the index of the non-zero entry in y_p.

$$g(\hat{y}_p, j_p) = \begin{cases} 1; \, if \, \hat{y}_p = j_p \\ 0; \, if \, \hat{y}_p \neq j_p \end{cases}$$

- Using this mapping, accuracy can be defined as:

$$A(\vec{w}) = \frac{1}{N} \sum_{p=1}^{N} g(\hat{y}_{0p}, j_p)$$

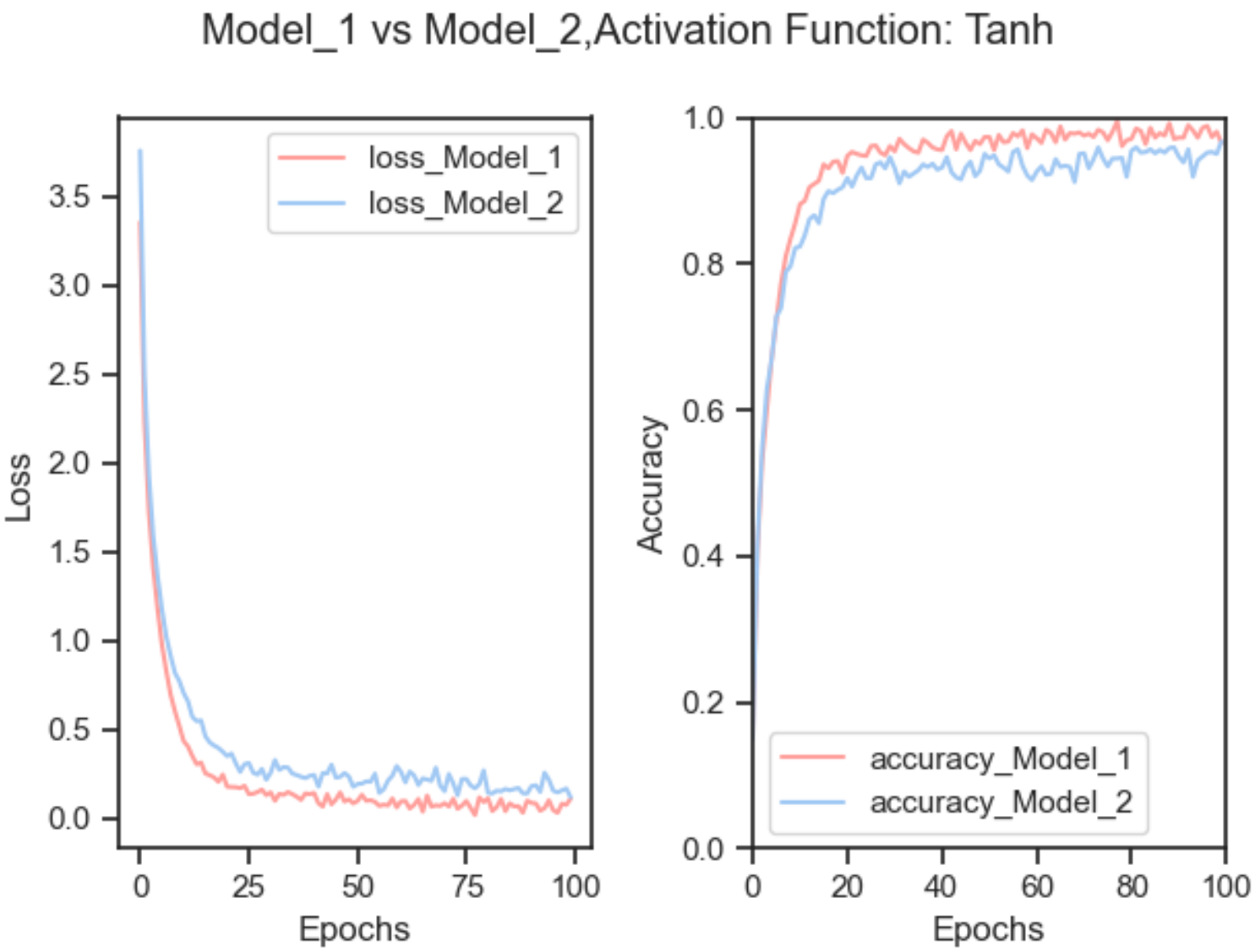
The accuracy of the model was calculated using the above measure.

- **Results:** We used 16 images from each of the 70 classes to train the model, with the remaining 4 used as test data.

Test Accuracy	0.6714285612106323
Test Loss	2.320704221725464

Affect of weight initialisation

- We initialised the weights of the matrix in different ways, and found that the convergence rate of the model is faster when the weights are initialised between the range of $\frac{-1}{n}$ to $\frac{1}{n}$ where n is the number of nodes in the layer.
- **Note:** The optimal weight initialization range can be analyzed by the convergence of change in error with respect to the weights (Which is a future scope of the project for research).



Model_1 has weights initialised between -1/n and 1/n.
Model_2 weights are initialised between -5 and 5.

Future Scope of project For Research

- The optimal weight initialization range can be analyzed by the convergence of change in error with respect to the weights. e
- Explore advanced adaptive learning rate strategies that dynamically adjust based on the gradient information.
- Investigate the use of basis other than standard basis for representing independent labels in optimization problems in multi class setting.

References

- Zhihong Ouyang, Lei Xue, Feng Ding, and Da Li. *An algorithm for extracting similar segments of moving target trajectories based on shape matching*. *Engineering Applications of Artificial Intelligence*, 127, 107243, 2024. https://doi.org/10.1016/j.engappai.2023.107243
- Charu C. Aggarwal. *Linear Algebra and Optimization for Machine Learning: A Textbook*. Springer Cham, 2020. https://doi.org/10.1007/978-3-030-40344-7
- Xavier Glorot and Y. Bengio. *Understanding the difficulty of training deep feedforward neural networks*. *Journal of Machine Learning Research - Proceedings Track*, 9, 249-256, 2010.[Link]